EXP NO: 4	ADVANCED SUPERVISED LEARNING TECHNIQUES

AIM:

To implement advanced supervised learning techniques such as Bagging, Random Forest, AdaBoost, Gradient Boosting, Voting and Stacking ensembles on the Breast Cancer dataset, and to compare their performance with a single Decision Tree.

ALGORITHM:

1. Import the required libraries and ensemble classes from scikit-learn.
2. Load the Breast Cancer dataset (binary classification).
3. Scale the features using StandardScaler and split the data into training and testing sets.
4. Define the models: Decision Tree (baseline), Bagging, Random Forest, AdaBoost, Gradient Boosting, Soft Voting and Stacking.
5. Train each model and predict the test data.
6. Evaluate each model using test accuracy, F1 score and 5-fold cross-validation accuracy.
7. Tabulate the results and plot a bar chart to compare the models.

CODE:

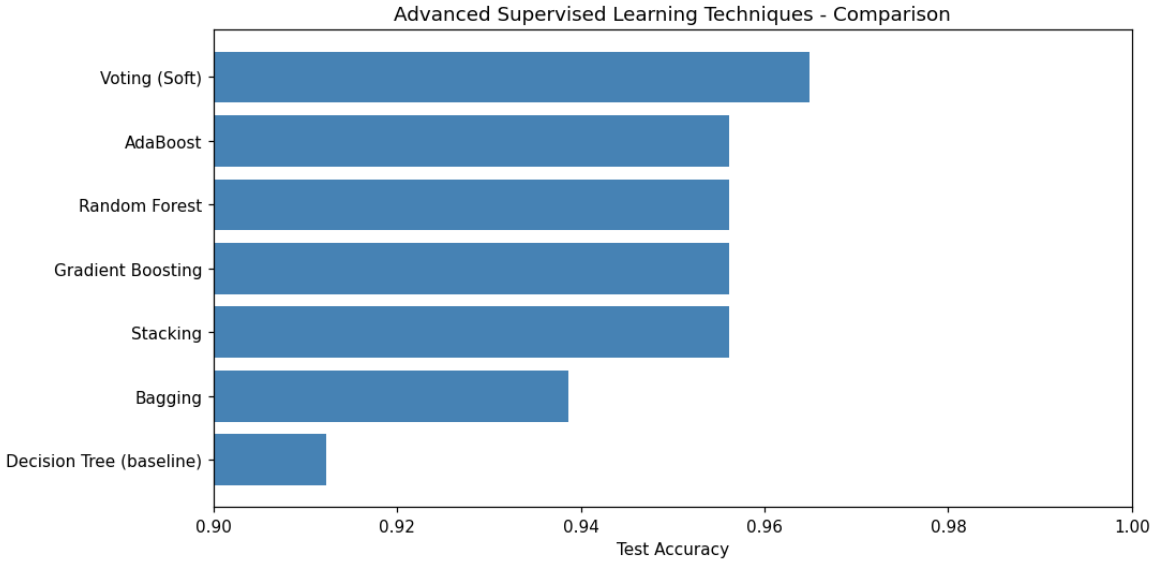
```
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import (BaggingClassifier, AdaBoostClassifier,
                              GradientBoostingClassifier,
                                  RandomForestClassifier,
                              VotingClassifier, StackingClassifier)
from sklearn.metrics import accuracy_score, f1_score

# 2. Load dataset (Breast Cancer - binary classification)
data = load_breast_cancer()
X, y = data.data, data.target
print("Dataset shape:", X.shape)
print("Classes:", data.target_names)
# 3. Scale and split
```

```
X_scaled = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y)

# 4. Define advanced supervised models
models = {
    "Decision Tree (baseline)": DecisionTreeClassifier(random_state=42),
    "Bagging": BaggingClassifier(estimator=DecisionTreeClassifier(),
                                 n_estimators=100, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "AdaBoost": AdaBoostClassifier(n_estimators=100, random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=100,
        learning_rate=0.1,
                                                    random_state=42),
    "Voting (Soft)": VotingClassifier(
        estimators=[('lr', LogisticRegression(max_iter=1000)),
                    ('knn', KNeighborsClassifier()),
                    ('rf', RandomForestClassifier(n_estimators=100,
                        random_state=42))],
        voting='soft'),
    "Stacking": StackingClassifier(
        estimators=[('rf', RandomForestClassifier(n_estimators=100,
            random_state=42)),
                    ('knn', KNeighborsClassifier())],
        final_estimator=LogisticRegression(max_iter=1000)),
}

# 5. Train, test and cross-validate every model
results = []
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    cv = cross_val_score(model, X_scaled, y, cv=5).mean()
    results.append([name, accuracy_score(y_test, pred), f1_score(y_test, pred),
        cv])

results_df = pd.DataFrame(results, columns=["Model", "Test Accuracy", "F1
    Score", "5-Fold CV Accuracy"])
results_df = results_df.sort_values("Test Accuracy",
    ascending=False).reset_index(drop=True)
print("\nModel Comparison:\n")
print(results_df.round(4).to_string())

# 6. Visual comparison
plt.figure(figsize=(10, 5))
plt.barh(results_df["Model"], results_df["Test Accuracy"], color='steelblue')
plt.xlim(0.9, 1.0)
plt.xlabel("Test Accuracy")
plt.title("Advanced Supervised Learning Techniques - Comparison")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

OUTPUT:

```
Dataset shape: (569, 30)
Classes: ['malignant' 'benign']

Model Comparison:
```

	Model	Test Accuracy	F1 Score	5-Fold CV Accuracy
0	Voting (Soft)	0.9649	0.9722	0.9754
1	AdaBoost	0.9561	0.9660	0.9772
2	Random Forest	0.9561	0.9655	0.9561
3	Gradient Boosting	0.9561	0.9660	0.9631
4	Stacking	0.9561	0.9660	0.9684
5	Bagging	0.9386	0.9517	0.9579
6	Decision Tree (baseline)	0.9123	0.9286	0.9173



RESULT:

Advanced ensemble techniques were successfully implemented. All the ensemble models performed better than the single Decision Tree (91.23%). The Soft Voting classifier gave the highest test accuracy of 96.49%, while AdaBoost gave the best 5-fold cross-validation accuracy of 97.72%.